

A Development Gain That Did Not Survive Locking: Negative and Inconclusive Results from an LLM-Based Mutation Test Generation Pipeline

Abstract—Supervised fine-tuning (SFT) of a small code LLM for mutation test generation produced a large apparent gain on the panel used to select its checkpoints, and a much smaller gain that did not meet a significance threshold fixed in advance on a locked validation split. On the selection panel, Kill@8 rose from 0.605166 to 0.695572. On the locked split the paired difference was +3.4346 percentage points (450/757 to 476/757) with an exact two-sided McNemar $p = 0.0783$, against a predeclared requirement of $p < 0.01$. Under the rule frozen before either arm ran, the untrained base model was retained. Over the same locked comparison, the fraction of reference-valid candidates fell by 11.8726 points while parse and execution validity did not degrade: the tuned model wrote more mutant-killing tests that more often failed against the correct implementation. We report these results as negative and inconclusive evidence respectively, and we are explicit that a non-significant difference is not evidence of no difference. We also report an evaluation-integrity failure. A one-time final measurement was authorized and consumed without producing any measurement, because a record-admission rule in the final-run path rejected records that the established evaluator had always excluded later by scope. The defect was reproducible on permitted splits and had never been executed against real data. The resulting safeguard — a receipt-bound, full-scale rehearsal on a permitted split — reproduced a previously measured value exactly, demonstrating pipeline execution fidelity without touching the consumed split. We distill the episode into actionable controls for anyone measuring LLM-based testing tools.

Index Terms—negative results, reproducibility, mutation testing, large language models, test generation, evaluation integrity, data leakage

I. INTRODUCTION

A measurement pipeline for LLM-based test generation has two ways to mislead. It can report a real number about the wrong population, and it can report a number that its own code never actually computed the way the paper says. This paper reports one instance of each, from a single project, with every figure traceable to an artifact frozen before the corresponding measurement ran.

The substantive result is negative in the specific sense the RENE track asks for. Supervised fine-tuning of a 1.5B-parameter code model for mutation test generation produced a +9.04 point Kill@8 gain on the panel that had been used to select its checkpoints, and +3.4346 points on a locked split that had not been used to select a checkpoint, tune a

prompt, or set a threshold. The locked difference did not meet a significance threshold fixed in advance, so the predeclared rule retained the untrained base model. The same comparison shows a substantial regression in candidate *reference validity* — whether a generated test passes against the correct implementation — with no accompanying regression in parsing or execution. A tuned model that kills more mutants while more often failing the reference is not obviously a better test generator, and a pipeline that reports only Kill@k would not have seen it.

The methodological result is an evaluation-integrity failure and its repair. A one-time final measurement, protected by a single-use authorization and a hash-bound receipt, was spent without producing any measurement. The immediate cause was a record-admission rule in the final-run path that required every record in a split to carry a non-empty set of fields, applied *before* the scope filter that the established evaluator had always applied first. Records legitimately carrying blank fields were therefore rejected rather than excluded. The defect reproduced on permitted splits at any time and cost nothing to find there; it survived because the component was deliberately unreachable until the authorization was spent, and so had never been executed against real data.

Contributions

- 1) A quantified instance of development-to-locked shrinkage in LLM-based mutation test generation: a +9.04 point selection-panel gain corresponding to +3.4346 points on a locked split, with the locked result failing a predeclared significance criterion (Section III).
- 2) Evidence that Kill@k and candidate reference validity moved in opposite directions under SFT, isolated from parse and execution validity (Section IV).
- 3) An analysis of how parsing semantics, retained raw outputs and split isolation determine whether a reported number can be interpreted at all (Section V).
- 4) A post-mortem of a consumed one-time measurement, and a receipt-bound full-scale rehearsal that reproduced a prior value exactly on a permitted split (Section VI).
- 5) A set of controls that would have caught each failure, stated as checks rather than principles (Section VIII).

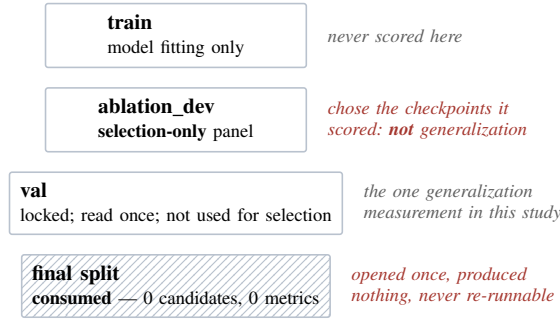


Fig. 1. Split roles. For the consumed final split we give neither its size, its membership, its record counts nor any performance score. The two zeros shown are a historical fact about the authorized attempt — it generated zero candidates and zero reportable metrics — not a measurement of the split. No result in this paper derives from it.

II. STUDY DESIGN

A. System under measurement

The measured system generates unit tests intended to distinguish a mutated function from its correct implementation. The generator is `Qwen2.5-Coder-1.5B-Instruct`, pinned at an immutable revision. All measurements reported here run under one named generation protocol, frozen and identified by digest: eight candidates per target, temperature 0.7, top- p 0.9, seed 42, whole-output candidate parsing, raw-output retention, a 1024 token completion limit, a 3072 token sequence limit, no reranking and no feedback rounds. Four arms were evaluated under one contract digest and one evaluation-scope digest, recorded as identical across all four.

The primary metric is Kill@ k : the fraction of target functions for which at least one of the first k generated candidates, in raw generation order, kills the mutant. We also report *parse validity* (the candidate is extractable and syntactically usable), *execution validity* (it runs without infrastructure error), and *reference validity* (it passes against the correct implementation). Reference validity is the quality signal most easily lost when only Kill@ k is reported, because a test that fails everything kills every mutant.

B. Splits and their roles

Figure 1 shows the four splits and, more importantly, what each one is permitted to support. The distinction that matters for this paper is between the panel that *selected* checkpoints and the split that was not used to select a checkpoint, tune a prompt, or set a threshold, and that determined the predeclared promotion decision.

C. Predeclared decision rule

Before either locked arm ran, a promotion rule was committed to a hash-bound receipt. Its first criterion required all three of: a Kill@8 gain of at least +3.0 points, a net gain of at least 15 functions, and an exact two-sided McNemar $p < 0.01$. Further criteria bounded the permitted regression in reference validity, parse and execution validity, and candidate diversity. The rule was fixed in advance precisely so that a

TABLE I
MAIN RESULTS. THE TWO BLOCKS ARE **NOT** COMPARABLE QUANTITIES: THE DEVELOPMENT BLOCK SELECTED THE CHECKPOINTS IT SCORES.

Panel	Arm	Kill@8	Fns	Wilson 95%
<i>Development — selection-only, $n = 542$</i>				
	base	0.605166	328	[0.5634, 0.6454]
	A@150	0.690037	374	[0.6499, 0.7275]
	A@431	0.695572	377	[0.6556, 0.7328]
	B@150	0.658672	357	[0.6178, 0.6973]
<i>Locked validation — not used for selection, $n = 757$</i>				
	base	0.594452	450	[0.5591, 0.6289]
	A@431	0.628798	476	[0.5938, 0.6625]
<i>Paired locked difference, A@431 vs. base</i>				
	gained/lost/net			+114 / -88 / +26
	Kill@8 delta			+3.4346 points
	McNemar exact p			0.0783
	predeclared requirement			$p < 0.01$ — not met
	decision			retain base model

borderline p could not be renegotiated afterwards, which is what a threshold is for.

III. RQ1: HOW MUCH DID THE SELECTION PANEL OVERSTATE?

Table I gives the result. On the selection panel the best arm reached $Kill@8 = 0.695572$ against a base of 0.605166, a gain of 9.04 points. On the locked split the same checkpoint reached 0.628798 against a base of 0.594452: +3.4346 points, a factor of roughly 2.6 smaller.

Two readings must be kept apart. The shrinkage is real and it is exactly what selection bias predicts: the development panel chose the checkpoint it then scored, so its figure is a selection statistic. The locked figure is the only generalization measurement in this study, and it is *positive*.

What the locked figure does not do is establish an improvement at the evidential standard set in advance. $p = 0.0783$ against a required $p < 0.01$ means the data are insufficient to conclude an effect at that standard. It does **not** mean the effect is absent, and we do not claim that [1], [2]. The honest summary is that the question is undetermined at this sample size, and it cannot now be resolved on this split: once a split has been used to make a decision, re-measuring it to obtain a different decision is no longer a held-out measurement.

Two further cautions follow from the design. First, $n = 757$ paired targets with 362 killed by both arms and 193 by neither leaves 202 discordant pairs, which is the effective sample for McNemar’s test [3]; with 114 versus 88 the test is simply not decisive. Second, a single paired observation of shrinkage on one task, one model and one corpus does not establish a general ratio, and we do not offer $2.6\times$ as one.

IV. RQ2: KILL@K AGAINST CANDIDATE VALIDITY

Table II shows a trade-off that a Kill@ k -only report would have hidden. Across the locked comparison, reference validity fell from 0.452114 to 0.333388, a change of -11.8726 points, and the number of target functions with *any* reference-valid candidate fell from 675 to 566. Over the same comparison,

TABLE II
CANDIDATE VALIDITY. PARSE AND EXECUTION DID NOT REGRESS, WHICH
MAKES THE REFERENCE-VALIDITY DROP SPECIFIC RATHER THAN
GENERAL DEGRADATION.

Panel	Arm	Parse	Exec.	Reference
<i>Development — selection-only</i>				
	base	0.983395	0.948570	0.565498
	A@150	0.997463	0.975092	0.458487
	A@431	0.998155	0.993081	0.455028
	B@150	0.998155	0.973478	0.448801
<i>Locked validation</i>				
	base	0.987285	0.928336	0.452114
	A@431	0.988771	0.942206	0.333388
	delta	+0.1486	+1.3870	-11.8726

parse validity rose slightly and execution validity rose by 1.3870 points.

The direction is consistent across all four development arms, so it is a property of this supervision family rather than of one checkpoint. The mechanism we can evidence is structural rather than semantic: a test that fails against every implementation kills every mutant it is scored against, so Kill@k rewards aggressiveness that reference validity penalises. Because parse and execution validity did not fall, the regression is not explained by the tuned model emitting more malformed or non-running code.

We are careful about the strength of this claim. We observed the trade-off; we did not manipulate it. Whether a different supervision recipe, a validity-aware objective or a reference-validity gate at generation time would remove it is unmeasured here.

V. RQ3: WHAT MAKES A REPORTED NUMBER INTERPRETABLE

Three pipeline properties determined whether the numbers above could be read at all, and each was, at some point, wrong in a way that produced plausible output.

a) Parsing semantics decide what was measured: The generator’s candidate extraction has two modes: taking the first assertion found, or taking the whole output. These score different artifacts. An earlier version of the final-run path never set the mode explicitly and inherited the legacy default, which would have produced a plausible number under one parser while the receipt claimed another. Parsing mode is not an implementation detail; it is part of the measurement’s definition, and it belongs in the frozen record alongside the seed.

b) Retained raw outputs are what makes a result auditable: Every candidate slot retains its raw model output and a digest of it. This is what allowed an independent recomputation of all 4,336 output digests in the rehearsal, and what allows a disputed score to be re-examined without regenerating anything. A pipeline that stores only verdicts cannot distinguish a scoring bug from a model behaviour.

c) Split isolation must be enforced by construction: Development, locked and final splits are only meaningfully

different if the code cannot reach the wrong one. In our case the loader that guarded the final split gated on *authorization* — a fact that, once recorded, never expires — rather than on whether anything remained to measure. This is a category error that any project using one-time measurements can make: a permission check is not an exhaustion check.

We also note a leakage concern specific to this task family. Where targets are drawn from public benchmarks, a model may reproduce a known assertion rather than derive one [4], [5]. The project separated assertion *form* from assertion *value* in classification for exactly this reason, so that a gain could not be silently credited to the wrong mechanism.

VI. RQ4: A REHEARSAL THAT REPRODUCES WITHOUT RE-MEASURING

A. What was lost

A one-time final measurement was authorized and attempted. It reached `failed_after_authorization`: the single-use authorization was spent and the run raised while admitting records, before generating anything. It produced **zero candidates and zero reportable metrics**. No prompt was rendered from a record of that split, nothing was generated, nothing was scored. The split is consumed and may never be rerun, because a split that has been opened is no longer held out regardless of what it produced.

The cause was narrow and entirely reproducible elsewhere. The final-run loader required a fixed set of record fields to be non-empty on *every* record in a split and raised before returning any of them. Records of one execution mode in this corpus legitimately carry some of those fields blank; the established evaluator had always loaded the whole split and *then* scoped scoring to the relevant mode. The final path inverted that order, so it refused exactly the records the pipeline was always going to exclude. Running the same selector against permitted splits reproduces the rejection immediately.

Critically, a repository-only exemption would still have been wrong: on the locked split three records of the *scored* mode also carry one of those fields blank, and they were measured. The requirement had to become mode-dependent, not merely tolerant of one record class.

B. Why the audits did not find it

The component had been reviewed repeatedly. Six generations of the readiness receipt were each superseded after an audit found a genuine defect, and a large test suite passed throughout. Every one of those defects was in something *recorded* but never *executed*. The pre-authorization smoke test proved the *generator* on two synthetic records; it did not prove the *loader* on a real split. The component was deliberately unreachable until the authorization was spent — and that design is precisely what kept it untested.

C. The rehearsal

The repair was not only to fix the admission rule but to remove the condition that made it untestable. We built a mode-aware admission function that partitions by execution

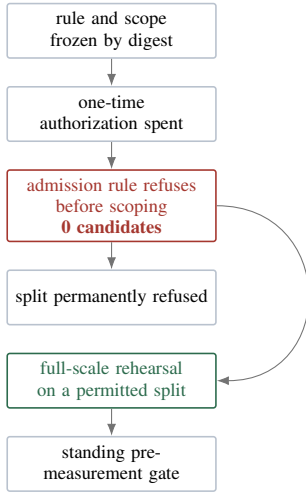


Fig. 2. From the consumed measurement to the safeguard. No record, identifier or payload from the consumed split appears in this paper.

mode first and applies field requirements only to the records that will actually be scored, importing the mode semantics from the established evaluator rather than restating them. We then ran the entire measurement path — receipt verification, admission, scoping, generation, whole-output parsing, safe execution, raw-output retention, progress writing and scoring — at full scale on a permitted split, under a hash-bound receipt.

The rehearsal scoped 594 requested records to 542 scored targets with 52 excluded by mode and zero admission errors, generated 4,336 candidates in 271 batches of two, retained 4,336 raw outputs with 4,336 digests, and reported zero missing and zero mismatched; an independent recomputation of every digest also found zero mismatches. It completed in 577.753 seconds with exit status 0, wrote no weights, loaded no adapter and performed no training. Its Kill@8 was 0.605166 — identical to the previously recorded value for the same model on the same split, under the same evaluation-scope digest.

That exact reproduction is the point, and its scope must be stated precisely. It is evidence that the rebuilt path computes the same number as the path that produced the historical value: *execution fidelity*. It is not a model-performance result, not a generalization estimate, not a selection result, and not a substitute for the measurement that was lost. The rehearsal receipt records `final_test_measurement: false`, `eligible_for_model_selection: false` and `supports_performance_claim: false` as machine-checkable fields, and the runner refuses any receipt that does not carry them.

VII. WHAT CAN AND CANNOT BE CLAIMED

Table III separates the three categories that this track exists to keep apart.

TABLE III

CLAIM STATUS. THE UNAVAILABLE BLOCK IS NOT A LIST OF FUTURE WORK; IT IS A LIST OF THINGS THIS STUDY CANNOT SUPPORT.

Established

Selection-panel Kill@8 rose 0.605166 → 0.695572; the same check-point gave 0.594452 → 0.628798 on the locked split. The paired locked difference was +3.4346 points, +114/−88, net +26, McNemar exact $p = 0.0783$. The predeclared criterion required $p < 0.01$; it was not met, and the base model was retained. Reference validity fell 11.8726 points on the locked comparison while parse and execution validity did not regress. A full-scale permitted-split rehearsal reproduced a prior Kill@8 of 0.605166 exactly, with complete raw-output integrity and no weights written. The one-time final measurement was consumed, producing zero candidates and zero metrics.

Inconclusive

Whether SFT improves Kill@8 on unseen data: undetermined at this sample size. The point estimate is positive; the predeclared threshold was not met. This is not evidence of no effect. Whether the reference-validity regression is intrinsic to this objective or an artifact of one recipe. Whether the observed shrinkage ratio generalises beyond this task, model and corpus.

Unavailable

No final-test result exists for any arm. No native repository-level kill rate exists; records of that mode are excluded from every rate reported here. No comparison against any external baseline tool exists; none was bundled or executed in any measurement reported here. Both held-out panels are expended: one on selection, one by the consumed attempt.

VIII. ACTIONABLE LESSONS

These are stated as checks because principles are easy to agree with and hard to fail.

- 1) **Freeze the decision rule, with its threshold, in a hashed artifact before any arm runs.** Our locked $p = 0.0783$ sat close enough to a boundary that a rule written afterwards would have been suspect whichever way it fell.
- 2) **Report at least one validity metric alongside Kill@k.** The regression we found is invisible to a kill-rate-only report and moves in the opposite direction to the headline.
- 3) **Bind sources by per-file digest, not a whole-tree hash.** A tree hash changes for reasons unrelated to the measurement and trains people to ignore it.
- 4) **Record both a raw and a line-ending-normalised source digest.** Otherwise an identical checkout on another platform appears to be different code.
- 5) **Make the scope filter run before the admission check, and derive both from one definition.** Two statements of the same rule drift; ours drifted into an unrecoverable failure.
- 6) **Fail the whole split, not the offending record.** A silently dropped target changes the denominator of every rate.

- 7) **Execute the full path on permitted data at full scale before any irreversible measurement.** A smoke test on synthetic records proves the generator, not the pipeline. This is the single control that would have prevented our loss outright.
- 8) **Make one-time authorizations two-phase.** Reserve on open, commit only once the first batch has been scored, so a failure after opening does not consume the split.
- 9) **Gate on exhaustion, not permission.** “Was this permitted once?” and “is there anything left to measure?” are different questions, and a recorded grant answers only the first.
- 10) **Test behaviour, not source text.** Several of our own checks matched strings and passed for the wrong reason; the durable versions parse syntax trees and sabotage file reads to prove a refusal precedes an access.

IX. THREATS TO VALIDITY

a) Construct validity: Kill@k rewards any candidate that distinguishes mutant from reference, including a test that fails universally — which is exactly why we report reference validity beside it. Mutants are a proxy for real faults, and the relationship is imperfect [6], [7].

b) Internal validity: One evaluation criterion in the locked run failed for a defect of our own: an adapter-specific digest had been placed inside a contract required to be identical across arms, which no base/adapter pair can satisfy. We record it as a failure rather than reinterpreting it. The decision does not rest on it, because the significance criterion failed independently. All arms ran under one contract digest and one scope digest, recorded as identical, which removes protocol drift as an explanation for the differences reported.

c) External validity: One model, one size, one corpus, one supervision family, one seed. We do not claim the shrinkage ratio, the validity trade-off, or their magnitudes transfer. Records of one execution mode are excluded from every rate, so no claim is made about performance on that class of target.

d) Statistical conclusion validity: The locked comparison is a single paired test on 202 discordant pairs. We report an exact two-sided McNemar test and Wilson intervals, and we deliberately avoid converting a non-significant result into a null claim [1], [8].

e) Reproducibility: The consumed split cannot be reproduced by anyone, including us. Everything else is reproducible from committed receipts under the frozen protocol; the rehearsal is direct evidence of that, having reproduced a prior value exactly.

X. RELATED WORK

a) Mutation testing: Mutation analysis is long established as a test-adequacy criterion [9], [10]. Its validity as a proxy for real faults has been studied directly, with mixed conclusions [6], [7]. Our contribution is not to mutation analysis itself but to how a mutation-based score behaves as an optimization target for a learned generator.

b) Automated test generation: Search-based and feedback-directed generators established the baseline for automated unit test construction [11]–[13]. LLM-based generators are now widely studied [14]–[16], with broad surveys of the area [17], [18].

c) Fuzzing and evaluation methodology: The fuzzing literature has confronted evaluation rigour directly: Klees *et al.* showed how easily fuzzing comparisons mislead without controlled seeds, trials and statistics [19], against a backdrop of widely used tools [20], [21] and, more recently, LLM-driven fuzzers [22], [23]. Our paper is in that tradition, applied to mutation-based test generation.

d) Leakage and reproducibility: Leakage is a recognised cause of overstated machine-learning results [5], benchmark contamination is documented for code models [4], [24]–[26], and reproducibility has been examined both across AI [27]–[29] and specifically for LLM use in software engineering [30]. Our distinctive contribution is an account of a reproducibility control that failed *in the direction of losing evidence* rather than fabricating it, and what it cost.

XI. CONCLUSION

An apparent development gain in LLM-based mutation test generation did not establish reliable improvement on a locked split, and candidate reference validity regressed substantially over the same comparison. Under a rule fixed before the measurement, the untrained base model was retained. Separately, a one-time final measurement was consumed without producing anything, because a component designed to be unreachable before authorization had never been executed against real data.

The controls that matter are unglamorous: freeze the rule, bind the sources, report validity beside the kill rate, and execute the whole path on permitted data at full scale before anything irreversible depends on it. The last of these is the one we learned by not having it.

DATA AVAILABILITY

An anonymized replication package accompanies this submission. It contains the frozen protocol and decision-rule receipts, the development and locked-validation result receipts, the rehearsal receipt and its sanitised execution receipt, the per-figure data files with the digest of their source receipt, and the exact commands to re-derive every table. Every numeric value in this paper is traceable to one of those receipts by digest. Raw generated outputs are retained but excluded from the package because they contain model completions in volume; their digests are published so any retained output can be verified. No material from the consumed split is included, and none exists.

REFERENCES

- [1] R. L. Wasserstein and N. A. Lazar, “The ASA statement on p-values: Context, process, and purpose,” *The American Statistician*, vol. 70, no. 2, pp. 129–133, 2016.
- [2] V. Amrhein, S. Greenland, and B. McShane, “Scientists rise up against statistical significance,” *Nature*, vol. 567, no. 7748, pp. 305–307, 2019.

- [3] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [4] S. Yang, W.-L. Chiang, L. Zheng, J. E. Gonzalez, and I. Stoica, "Rethinking benchmark and contamination for language models with rephrased samples," arXiv preprint arXiv:2311.04850, 2023.
- [5] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.
- [6] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2014, pp. 654–665.
- [7] M. Papadakis, D. Shin, S. Yoo, and D.-H. Bae, "Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 537–548.
- [8] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [9] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [10] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*. Elsevier, 2019, vol. 112, pp. 275–378.
- [11] G. Fraser and A. Arcuri, "Evosuite: Automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, 2011, pp. 416–419.
- [12] —, "Whole test suite generation," *IEEE Transactions on Software Engineering*, vol. 39, no. 2, pp. 276–291, 2013.
- [13] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for java," in *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA)*, 2007, pp. 815–816.
- [14] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 919–931.
- [15] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.
- [16] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers and focal context," arXiv preprint arXiv:2009.05617, 2020.
- [17] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.
- [18] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, 2024.
- [19] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 2123–2138.
- [20] K. Serebryany, "Continuous fuzzing with libfuzzer and addresssanitizer," in *IEEE Cybersecurity Development (SecDev)*, 2016, p. 157.
- [21] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based grey-box fuzzing as markov chain," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 1032–1043.
- [22] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.
- [23] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 423–435.
- [24] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
- [25] M. Chen, J. Twarek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.
- [26] J. Austin, A. Odena, M. Nye, M. Bosma *et al.*, "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.
- [27] O. E. Gundersen and S. Kjensmo, "State of the art: Reproducibility in artificial intelligence," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [28] M. Hutson, "Artificial intelligence faces reproducibility crisis," *Science*, vol. 359, no. 6377, pp. 725–726, 2018.
- [29] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-303.html>
- [30] J. Sallou, T. Durieux, and A. Panichella, "Breaking the silence: The threats of using llms in software engineering," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2024, pp. 102–106.